

ASDR – FIRST e BNF-Estendida

Fabio Lubacheski
fabio.lubacheski@mackenzie.br

Analizador sintático descendente recursivo - ASDR

- Na aula passada, vimos que para implementar um Analisador Sintático Descendente Recursivo Preditivo – ASDR, as restrições abaixo devem ser atendidas.
 - **Não pode ocorrer dúvida** na escolha de qual produção será aplicada.
 - A gramática **não pode conter recursividade à esquerda**.
 - A gramática deve estar **fatorada**.

Analizador sintático descendente recursivo - ASDR

- Para que não ocorra dúvida na escolha de qual produção será escolhida na derivação, as produções da gramática precisam atender as restrições abaixo:
 1. Toda produção é da forma $A ::= y\alpha$, onde y é um terminal.
 2. Se $A ::= y_1\alpha_1 \mid y_2\alpha_2 \mid \dots \mid y_n\alpha_n$ são todas as alternativas para o não-terminal A , então os terminais y_i são todos diferentes entre si.
- As restrições impostas acima são muito severas para serem usadas na prática, para torná-las menos restritivas utilizaremos o conceito de **FIRST** (primeiros símbolos).

FIRST

- Seja α é um símbolo da gramática (terminal ou não-terminal) o $\text{FIRST}(\alpha)$ é conjunto de símbolos terminais deriváveis a partir de α .
- Para calcular o **FIRST(α)** aplicam-se as regras:
 1. Se α for um **terminal**, então **FIRST(α)** é $\{\alpha\}$.
 2. Se $\alpha ::= \lambda$ for uma produção, **adiciona λ ao FIRST(α)**.
 3. Se α for um não-terminal e $\alpha ::= y_1 y_2 \dots y_n$ uma produção, colocar em **FIRST(α)** tudo o que estiver em **FIRST(y_1)**.
 - I. Se y_1 não derivar λ , então **não acrescentamos mais nada ao FIRST(α)**.
 - II. Se y_1 derivar λ , então é adicionado **FIRST(α)** o **FIRST(y_2)**, e assim por diante.
 - III. λ **é adicionado ao FIRST(α)**, se forem adicionados sucessivamente os **FIRST(y_1)**, **FIRST(y_2)**, ..., **FIRST(y_{n-1})** e **FIRST(y_n)** e y_n derivar λ .

FIRST

- Assim a limitação apresentada anteriormente para escrita de uma gramática para o ASDR

1. Toda produção é da forma $A ::= y\alpha$, onde y é um **terminal**.

2. Se $A ::= y_1\alpha_1 \mid y_2\alpha_2 \mid \dots \mid y_n\alpha_n$ são todas as alternativas para o não-terminal A , então os terminais y_i são todos diferentes entre si.

- Deverá ser modificada para **somente** limitação abaixo:

Se $A ::= y_1\alpha_1 \mid y_2\alpha_2 \mid \dots \mid y_n\alpha_n$ são todas as alternativas para o não-terminal A , então os conjuntos de **FIRST**(y_i) são disjuntos dois a dois.

Assim y_i pode ser um **terminal** ou **não-terminal**.

Exercício

1) Considere a gramática com símbolo inicial S:

$$S ::= AS \mid BA$$
$$A ::= aB \mid C$$
$$B ::= bA \mid d$$
$$C ::= c$$

a) Construa a árvore de derivação para a sentença “abaddc”.

b) Calcule o **FIRST** para os não-terminais da gramática.

c) Implemente o ASDR para a gramática utilizando os **FIRST** dos não-terminais.

d) Teste o seu ASDR para sentença “abaddc”.

Exercício

2) Considere a gramática com símbolo inicial S:

$S ::= ABCDd$

$A ::= aA \mid \lambda$

$B ::= bC \mid \lambda$

$C ::= cD \mid \lambda$

$D ::= e$

- a) Apresente o **FIRST** de cada um dos não-terminais da gramática
- b) Implemente o ASDR para a gramática utilizando os **FIRST** dos não-terminais.
- c) Teste o analisador sintático para uma cadeia com no mínimo 5 caractere.

Exercício

2) Considere a gramática com símbolo inicial E:

$$E ::= TA$$
$$A ::= +TA \mid \lambda$$
$$T ::= FB$$
$$B ::= *FB \mid \lambda$$
$$F ::= (E) \mid a$$

- a) Apresente o **FIRST** de cada um dos não-terminais da gramática
- b) Implemente o ASDR para a gramática utilizando os **FIRST** dos não-terminais.
- c) Teste o analisador sintático para uma cadeia com no mínimo 5 caractere.

Analizador sintático descendente recursivo - ASDR

- Já conseguimos flexibilizar o primeiro requisito para que seja possível implementar um ASDR, agora vamos ver como eliminar e fatorar uma gramática:
- **Não pode ocorrer dúvida** na escolha de qual produção será aplicada.
- A gramática **não pode conter recursividade à esquerda**.
- A gramática deve estar **fatorada**.

Recursividade à esquerda

- Considere a gramática livre de contexto abaixo:

$$E ::= E + T \mid T$$
$$T ::= a \mid b \mid c$$

- Será que conseguimos escrever um **ASDR** para gramática ?

Recursividade à esquerda

- Considere a gramática livre de contexto abaixo:

$$E ::= E + T \mid T$$
$$T ::= a \mid b \mid c$$

- Será que conseguimos escrever um **ASDR** para gramática ?
NÃO, POIS A GRAMÁTICA É RECURSIVA À ESQUERDA

Eliminação de recursividade à esquerda

- A **recursividade à esquerda** ocorre quando o símbolo mais à esquerda do lado direito da produção é o mesmo que o não-terminal que define a produção.

Considere a gramática

E ::= **E** + T | T

Note que com essa produção podemos ter derivações da seguinte forma:

$E \Rightarrow T$, $E \Rightarrow E+T \Rightarrow T+T$, $E \Rightarrow E+T \Rightarrow E+T+T \Rightarrow T+\dots+T+T$

Isto é, podemos ter a seguinte derivação sucessiva

$E \Rightarrow^* T+\dots+T+T+T$

Eliminação de recursividade à esquerda

- Podemos derivar um número arbitrário de $\{+T\}^*$, Este fato sugere a substituição da **notação recursiva** por uma **notação iterativa**, ou seja, reescrever a gramática em uma notação que permita implementar o **reconhecedor**.
- Essa notação recebe o nome de BNF estendida ou **EBNF** (Formalismo de *Backus-Naur* Estendido),
- Assim a construção $\{ \alpha \}$ numa produção denotará a repetição da **cadeia** α zero ou mais vezes (α^*).
- De maneira geral temos um produção na seguinte forma:

$$A ::= y_1 \mid y_2 \mid \dots \mid y_n \mid A\alpha$$

Pode ser reescrita por:

$$A ::= (y_1 \mid y_2 \mid \dots \mid y_n)\{\alpha\}$$

Eliminação de recursividade à esquerda

- Como ficaria a gramática abaixo sem recursividade à esquerda ?

$E ::= E + T \mid T$

$T ::= a \mid b \mid c$

Eliminação de recursividade à esquerda

- Como ficaria a gramática abaixo sem recursividade à esquerda ?

$E ::= E + T \mid T$

$T ::= a \mid b \mid c$

Gramática sem recursividade à esquerda

$E ::= T \{ + T \}$

$T ::= a \mid b \mid c$

- **Elimine a recursividade à esquerda para a gramática abaixo.**

$A ::= b \mid c \mid Aa$

Fatoração

- A Fatoração à esquerda é uma transformação gramatical útil para criação de uma gramática adequada ao reconhecedor. A ideia é **quando não estiver claro qual das duas produções alternativas escolher**.
- A forma geral para aplicar a fatoração é:
 $A ::= \beta y_1 \mid \beta y_2 \mid \beta y_3 \mid \dots \mid \beta y_n$ com $\beta \neq \lambda$. (β diferente de vazio)

pode-se fatorar essas produções escrevendo:

$$A ::= \beta(y_1 \mid y_2 \mid y_3 \mid \dots \mid y_n)$$

- Caso $y_i = \lambda$ para algum i , coloca-se esta alternativa em último lugar para implementar. **Analogamente podemos aplicar a fatoração à direita**, para simplificar uma produção.

Fatoração à esquerda

- Fatore à esquerda a gramática abaixo.

$A ::= ac \mid ab \mid a$

Fatoração à esquerda

- Fatore à esquerda a gramática abaixo.

$A ::= ac \mid ab \mid a$

Nesse exemplo $\beta=a$ e $y_1=c$, $y_2=b$, $y_3=\lambda$, assim teríamos:

$A ::= a(c|b|\lambda)$

- Nas notação **EBNF** temos a construção $[\alpha]$ que é equivalente a $\alpha|\lambda$, ou seja, indicará que a cadeia α é opcional. Reescrevendo o exemplo acima temos:

$A ::= a[c|b]$

Fatoração à esquerda

- Como ficaria a gramática abaixo sem recursividade à esquerda e fatorada ?

$E ::= E + T \mid E - T \mid T$

$T ::= \text{num}$

Exercícios

Nas implementações a seguir considere que o símbolo inicial para ser avaliado pelo analisador sintático estará armazenado na variável **char lookahead** e a função **void consome(char atomo)** está implementada e é responsável em atualizar **lookahead**.

1) Considere a gramática abaixo na notação **BNF** com símbolo inicial S:

$S ::= (L) \mid a$

$L ::= L , S \mid S$

a) Encontre as árvores gramaticais para as seguintes sentenças:

i. (a,a)

ii. (a,(a,a))

iii. (a,((a,a),(a,a)))

b) Reescreva a gramática na notação EBNF para que seja possível implementar um ASDR (sem recursividade à esquerda).

c) Implemente um ASDR para essa gramática

Exercícios

2) Considere a gramática abaixo na notação **BNF** com símbolo inicial A:

$$A ::= B * A \mid B$$
$$B ::= B > C \mid C$$
$$C ::= E \mid \sim E$$
$$E ::= a \mid (A)$$

a) Encontre a árvore de derivação para sentença: $a > a^*(\sim a^*a)$

b) Rescreva as produções, em notação **BNF estendida** conveniente (sem recursividade à esquerda e fatorada).

c) Implemente o um reconhecedor para gramática na notação estendida.

Exercícios - EBNF

3) Considere a gramática abaixo na notação BNF com símbolo inicial <expressão>, observe que nessa gramática os <não-terminais> são nomes entre parênteses angulares < e > e os símbolos **terminais** estão em **apostrofo** ou **negrito**.

<expressão> ::= <expressão> '+' <termo> | <expressão> '-' <termo> | <termo>

<termo> ::= <termo> '*' <fator> | <termo> '/' <fator> | <fator>

<fator> ::= 'a' | 'b' | 'c' | ... | '1' | '2' | '3' | ... | '(' <expressão> ')'

a) Rescreva as produções, em notação **BNF estendida** conveniente (sem recursividade à esquerda e fatorada).

b) Implemente um ASDR para essa gramática na notação estendida.

Exercício

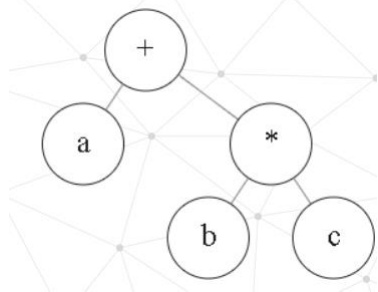
- 4) Considere a gramática abaixo, como você reescreveria a gramática de tal forma que fosse possível escrever expressões aritméticas com o operador de multiplicação ?

$E ::= E + T \mid T$

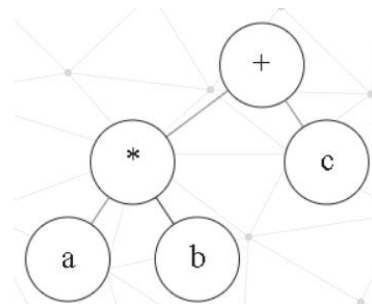
$T ::= a \mid b \mid c$

Lembrando que o operador de multiplicação deve ser sempre resolvido antes em um expressão aritmética, ou seja, na árvore de expressão o operador deve ficar mais próximo as folhas.

$a+b*c$



$a*b+c$



Veja o link: <https://Inoqueir.github.io/expression-tree-gen/>

Dúvidas ?

Bons estudos !!!